

Received 30 June 2026, accepted 11 July 2026, date of publication 14 July 2026, date of current version 24 July 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3713401

RESEARCH ARTICLE

Self-Evolving AI Agents With Dual Memory for Automated Software Testing and Bug Localization

TIANJUN MO¹, CHANGHAO ZHANG², JITONG ZOU³, ZIXUAN GUO⁴, AND MINJAE RHEE⁵

¹Duke University, Durham, NC 27708, USA

²The University of Chicago, Chicago, IL 60637, USA

³Case Western Reserve University, Cleveland, OH 44106, USA

⁴New York University, New York, NY 10012, USA

⁵University of Illinois Urbana-Champaign, Champaign, IL 61820, USA

Corresponding author: Minjae Rhee (minjaer3@illinois.edu)

ABSTRACT Large Language Model (LLM)-based autonomous agents have shown significant promise in automating software engineering tasks, yet existing systems still suffer from two fundamental limitations: i) the lack of persistent experiential knowledge across debugging sessions, which forces agents to repeat exploratory mistakes, and ii) the static nature of prompt structures, which prevents agents from adapting their reasoning strategy to recurring failure modes. To address these gaps, we propose **LS-CM** (Long-Short Collaborative Memory), a dual-memory architecture that couples a Retrieval-Augmented Generation (RAG)-based short-term memory for code-base context with a Reinforcement Learning (RL)-driven long-term memory that distills strategic patterns from historical test failures and patch iterations. The core innovation is a self-evolving context engineering module that automatically refines the agent's internal prompt structure based on execution feedback signals, treating the prompt as a configurable policy object that a gradient-free controller selects among pre-generated variants, rather than a fixed artifact. We evaluate LS-CM on two industry-standard benchmarks: SWE-bench, for end-to-end issue resolution on real-world GitHub repositories, and Defects4J, for bug localization and program repair in Java projects. Across multiple LLM backbones, LS-CM resolves 33.7% of SWE-bench Lite issues, a modest gain over the strongest reproduced baseline (Agentless, 32.0%), and improves Top-1 fault localization accuracy on Defects4J from 42.6% to 47.6% while reducing editing churn by 31%. We find that LS-CM's most robust advantages lie in debugging efficiency, cross-episode strategy transfer, and run-to-run stability rather than in a single headline resolved-rate number. Ablation studies confirm that short-term retrieval, long-term policy learning, and self-evolving prompts each contribute to the gains. Our work demonstrates that explicitly modeling memory and prompt adaptation as first-class learning objects substantially enhances agent productivity on real-world software engineering tasks.

INDEX TERMS Autonomous agents, large language models, retrieval-augmented generation, software testing, bug localization, prompt engineering.

LIST OF ABBREVIATIONS

The following abbreviations are used throughout this paper.

Acronym	Expansion and context
LLM	Large Language Model: the frozen backbone (e.g., GPT-4o) that executes actions.

The associate editor coordinating the review of this manuscript and approving it for publication was Ahmadreza Montazerolghaem¹.

LS-CM	Long-Short Collaborative Memory: the architecture proposed in this paper.
RAG	Retrieval-Augmented Generation: grounding generation in retrieved context; basis of STM.
RL	Reinforcement Learning: used to train the long-term strategy policy.
STM	Short-Term Memory: the hybrid code-retrieval module (Section IV).
LTM	Long-Term Memory: the RL-trained strategy-selection policy (Section IV).

SECE	Self-Evolving Context Engineering: the bandit-driven prompt-configuration controller.
PPO	Proximal Policy Optimization: the RL algorithm used to train the LTM policy.
LinUCB	Linear Upper Confidence Bound: the contextual-bandit rule used by SECE.
GAE	Generalized Advantage Estimation: the advantage estimator used by PPO.
BM25	Best Matching 25: a sparse lexical retrieval function used inside STM.
MLP	Multi-Layer Perceptron: the lightweight network parameterizing the LTM policy.
FL	Fault Localization: identifying the buggy code element.
APR	Automated Program Repair: producing a passing patch.
MFR	Mean First Rank: average rank of the first correctly localized fault.
CoT	Chain-of-Thought: step-by-step prompting.
DPO	Direct Preference Optimization: an alignment method discussed in future work.
MCP	Model Context Protocol: a standardized tool/context interface discussed in future work.
CI/CD	Continuous Integration / Continuous Delivery.
API	Application Programming Interface.

I. INTRODUCTION

Software testing and bug localization are among the most labor-intensive and economically consequential activities in modern software development. Industry studies consistently report that developers spend 35–50% of their time diagnosing and fixing defects [1], [2], motivating decades of research into automated techniques [3]. We emphasize at the outset that “software testing” comprises many distinct activities, including test-case generation, test prioritization, coverage analysis, regression testing, and defect diagnosis. This paper targets the diagnosis-and-repair portion of that spectrum, namely repository-level fault localization and automated program repair. We do not address test generation or other testing sub-disciplines, and we narrow our claims accordingly throughout (see the scope note at the end of this section). The recent emergence of Large Language Models (LLMs) such as GPT [4], Codex [5], Code Llama [6], and StarCoder [7] has fundamentally reshaped this landscape: equipped with reasoning capabilities derived from chain-of-thought prompting [8] and tool use via ReAct-style frameworks [9], LLM agents can now navigate real repositories, execute commands, and propose patches at unprecedented levels of autonomy.

Despite this progress, state-of-the-art coding agents, ranging from SWE-agent [10] to Agentless [11] and multi-agent collaboration frameworks like MetaGPT [12] and ChatDev [13], still exhibit two persistent shortcomings. First, they operate largely as memoryless systems: each

new bug is approached *de novo*, with no mechanism for transferring lessons from prior debugging episodes. Even when retrieval-augmented generation [14], [15] is used to fetch repository context, the retrieved snippets are episodic and short-lived, lacking strategic abstraction. Second, the agent’s prompt scaffolding, including its system instructions, tool descriptions, and reasoning templates, is typically hand-crafted and static, treated as immutable infrastructure rather than as a learnable component. Recent attempts to introduce reflection [16] or self-refinement [17] provide local feedback within an episode, but do not yield durable improvements that persist across tasks.

Cognitive science suggests that human experts exhibit precisely the capabilities current agents lack: they hold transient working memory of the immediate problem while drawing on long-term semantic and procedural memory of strategies acquired through years of experience. This dual-memory organization is widely viewed as a substrate for adaptive expertise. Inspired by this view, we ask: can an LLM agent be endowed with an analogous dual-memory mechanism, and can its prompt structure itself adapt from feedback?

We answer this question affirmatively by proposing **LS-CM (Long-Short Collaborative Memory)**, a unified architecture for self-evolving software engineering agents. LS-CM jointly leverages two memory subsystems and a prompt-adaptation controller:

- A **short-term memory (STM)** module implements hybrid dense–sparse retrieval over the current code base, dynamically supplying the most relevant files, call-graph slices, and test outputs to the agent’s context window. STM is built on dense retrieval techniques [15] augmented with BM25 [18] for symbol-level recall, and uses CodeBERT [19] and GraphCodeBERT [20] embeddings for semantic matching.
- A **long-term memory (LTM)** module stores parameterized strategic patterns extracted from historical experiences. We train a policy network using Proximal Policy Optimization [21] on rewards derived from test-suite outcomes, enabling LTM to recommend action sequences (e.g., “first inspect stack traces, then localize via spectrum methods [22]”) most likely to succeed for the current bug class.
- A **self-evolving context engineering (SECE)** controller treats the agent’s prompt as a parameterized object. After each episode, SECE updates prompt components: tool descriptions, exemplar selection, and reasoning instructions, using a credit-assignment mechanism that connects prompt segments to observed reward signals. This generalizes the prompt-as-program perspective of DSPy [23] into a feedback-driven loop.

The interaction between STM, LTM, and SECE forms a tightly integrated loop: STM grounds reasoning in current context, LTM biases exploration toward historically successful strategies, and SECE re-engineers the prompt envelope itself when LTM detects systematic failure modes.

While RAG-based retrieval, RL-based policy learning, and prompt optimization each exist independently in the literature, the novelty of LS-CM lies in their integration under a single execution-grounded reward, and in particular in the SECE formulation that couples feedback-driven prompt configuration to a persistent, cross-episode strategy memory. We are explicit about what we do not claim: the retriever is fixed, and the LTM policy and the SECE controller are trained by decoupled, alternating updates rather than by a single joint end-to-end objective (Section IV). To our knowledge, LS-CM is nonetheless the first repository-level software-engineering agent to combine a persistent cross-episode strategy memory with feedback-driven prompt configuration, evaluated on real issue-resolution benchmarks.

Our principal contributions are summarized as follows:

- 1) We introduce **LS-CM**, a dual-memory agent architecture that explicitly separates short-term contextual retrieval and long-term strategic abstraction for repository-level fault localization and program repair.
- 2) We formulate **self-evolving context engineering** as a credit-assigned prompt-configuration problem and provide an efficient gradient-free update rule that requires no LLM fine-tuning.
- 3) We conduct extensive experiments on **SWE-bench** [24] and **Defects4J** [25], reconciling all baseline numbers with the published record, and showing that LS-CM consistently outperforms strong baselines including SWE-agent [10], Agentless [11], and Reflexion [16], with detailed ablations isolating the contribution of each module.
- 4) We analyze the limitations of LS-CM and discuss its broader applicability to adjacent domains such as security vulnerability discovery, autonomous data science, and scientific experimentation.

A. SCOPE AND TERMINOLOGY

To preclude over-reading, we fix three terms. (1) By short-term memory we mean a query-conditioned, per-episode working buffer of retrieved code and execution observations; it is implemented as a specialized retrieval mechanism and we defend the “memory” label, with caveats, in Section IV. (2) By long-term memory we mean a procedural (parametric) memory that distills cross-episode experience into policy weights; it does not store or replay raw past episodes, and is therefore closer to learned strategy selection than to episodic recall. (3) By self-evolving we mean closed-loop selection among a fixed catalog of pre-generated prompt variants driven by execution feedback; genuine on-the-fly prompt generation is deferred to future work. Readers who prefer a more conservative framing may read LS-CM as a hybrid retrieval, strategy-selection, and prompt-configuration approach for repository-level automated repair; the empirical results are unchanged under either reading.

The remainder of the paper is organized as follows. Section II reviews related work on LLM-based coding agents, retrieval augmentation, memory architectures, and reinforcement learning for code. Section III introduces

preliminaries. Section IV details the LS-CM architecture and its mathematical formulation. Section V presents experimental setup and results. Section VI concludes with limitations and future directions.

II. RELATED WORKS

A. LLM-BASED SOFTWARE ENGINEERING AGENTS

The first generation of LLM coding tools focused on autoregressive code generation, exemplified by Codex [5], Code Llama [6], and StarCoder [7], building on representation-learning advances such as CodeBERT [19] and GraphCodeBERT [20]. Subsequent work extended these models with execution-aware augmentations such as test-driven candidate filtering [26]. More recently, AutoML-Pipeline studies RAG-enhanced code generation for cloud-native machine-learning workflows, using a pre-validation agent to check resource and dependency constraints before deployment [27].

A second wave reframed coding as agentic decision making. ReAct [9] introduced an interleaved reasoning-acting paradigm; Reflexion [16] added verbal self-reinforcement; and Self-Refine [17] formalized iterative self-feedback. SWE-agent [10] demonstrated that carefully engineered agent-computer interfaces can dramatically improve issue-resolution rates, while Agentless [11] showed that simple pipelines can rival complex agents. Multi-agent collaboration frameworks such as MetaGPT [12] and ChatDev [13] further partition responsibilities among specialized roles. We stress that MetaGPT and ChatDev target end-to-end code generation from requirements. Their goal is to turn a natural-language specification into a working program from scratch, which is fundamentally different from locating and repairing defects in a pre-existing repository. We cite them as representative multi-agent SE frameworks (and, in future work, as candidate substrates for a shared memory), not as direct debugging or repair baselines; none of our quantitative comparisons in Section V are against them. A recent survey [28] provides a comprehensive overview of this trajectory.

None of these systems, however, treats memory or prompt structure as objects to be learned from accumulated experience. Our work directly targets this gap.

B. MEMORY-AUGMENTED AND RETRIEVAL-AUGMENTED AGENTS

Retrieval-Augmented Generation (RAG) [14] has become a standard tool for grounding LLMs, supported by dense retrievers [15] and classical sparse methods [18]. In agentic settings, MemGPT [29] models memory hierarchically, MemoryBank [30] introduces long-term storage with Ebbinghaus-style forgetting, and Generative Agents [31] popularize reflective memory streams. Voyager [32] demonstrates a skill library that accumulates over time in open-ended environments. While these systems illustrate the value of persistent memory, their application to real-world

software engineering, with its strict correctness constraints and complex execution semantics, remains underexplored.

C. REINFORCEMENT LEARNING FOR CODE AND AGENTS

Reinforcement learning has been applied to code generation through execution-based rewards. CodeRL [33] pioneered actor-critic optimization on unit-test outcomes. RLHF-style alignment [34], [35] and PPO [21] provide the algorithmic foundation. Agent-level RL is, however, more challenging due to long horizons and sparse rewards. AgentBench [36] highlights the current performance gap. LATS [37] integrates tree search with reasoning. Our LTM module builds on this line by learning strategic policies over symbolic actions rather than token-level distributions, which avoids the prohibitive cost of fine-tuning large coding models.

D. FAULT LOCALIZATION AND AUTOMATED PROGRAM REPAIR

Classical fault localization includes spectrum-based methods like Ochiai [22]. Learning-based extensions such as DeepFL [38] and coverage-representation methods [39] have considerably advanced Top- k accuracy. For automated program repair, InferFix [40] and ChatRepair [41] demonstrated strong results using LLMs with iterative dialogue. We adopt Defects4J [25] as a canonical evaluation suite and complement these methods with our memory-augmented agent.

E. PROMPT OPTIMIZATION AND AUTOMATIC PROMPT ENGINEERING

A growing body of work explores prompt structure as a learnable artifact. DSPy [23] compiles modular prompt pipelines, while in-context learning analyses reveal the sensitivity of LLMs to exemplar ordering. Contextual-bandit methods such as LinUCB [42] provide a principled, gradient-free way to select among discrete options under uncertainty, and we adopt this machinery for prompt selection. To date, however, prompt optimization has rarely been integrated with execution-based feedback in agentic systems. Our SECE controller fills this niche by tying prompt selection to long-term reward signals.

III. PRELIMINARIES

A. NOTATION

Table 1 summarizes the mathematical symbols used throughout the paper, with the equation or section in which each is introduced. We collect them here so that readers need not hunt for definitions scattered across the text.

B. PROBLEM FORMULATION

We formalize automated software testing and bug localization as a sequential decision problem. Given a code repository $\mathcal{R}$ and an issue specification q (e.g., a GitHub issue or a failing test), an agent must produce a patch $\hat{p}$ such that the patched repository $\mathcal{R} \oplus \hat{p}$ passes a held-out test suite $\mathcal{T}_q$. At each step t , the agent observes state $s_t = (\mathcal{R}_t, q, h_t)$,

TABLE 1. Summary of notation.

Symbol	Meaning
$\mathcal{R}$	Code repository; $\mathcal{R} \oplus \hat{p}$ is $\mathcal{R}$ with patch $\hat{p}$ applied.
q	Issue specification (GitHub issue or failing test).
$\hat{p}$	Candidate patch produced by the agent.
$\mathcal{T}_q$	Held-out test suite for issue q .
s_t	State at step t : $(\mathcal{R}_t, q, h_t)$.
h_t	Interaction history up to step t .
a_t	Action at step t (inspect/edit/run/test).
$\mathcal{A}$	Discrete action space.
$T(\cdot)$	Deterministic transition function.
r_{term}	Binary terminal reward (Eq. 1).
r_t	Per-step shaped reward (Eq. 3).
γ	Discount factor.
$\pi_\theta(a \mid s)$	Generic policy with parameters θ .
$J(\theta)$	Expected discounted return.
$\phi(\cdot)$	Learned retrieval similarity (Eq. 2).
$\mathcal{D}_k$	Top- k retrieved documents.
λ	Dense/sparse mixing coefficient in STM.
E_ψ	GraphCodeBERT-based encoder.
σ, Σ	A strategy template and the strategy bank ($ \Sigma = 24$).
π_θ^σ	LTM strategy policy (a 3-layer MLP).
$\hat{A}_t$	Generalized advantage estimate.
$\rho_t(\theta)$	PPO importance ratio (Eq. 4).
ϵ	PPO clipping parameter.
θ_P	Active prompt configuration index in SECE.
$R(\theta_P)$	Expected episode reward under θ_P .
κ	LinUCB exploration coefficient (Eq. 6).
ΔR_c	Counterfactual credit of prompt component c (Eq. 7).
α, β, η	Reward-shaping weights (Eq. 3).
m, k	#strategies / #snippets surfaced at inference.
λ_{GAE}	GAE smoothing parameter.
β_H	Entropy-bonus coefficient.
$\xi(q)$	Bug-context feature map (defined in Section V).

where h_t is the interaction history, and selects an action $a_t \in \mathcal{A}$ from a discrete action space comprising file inspection, code editing, command execution, and test invocation. The transition dynamics $s_{t+1} = T(s_t, a_t)$ are deterministic given the underlying file system and test runner. The terminal reward is binary:

$$r_{\text{term}} = \mathbb{I}[\mathcal{R}_T \oplus \hat{p} \models \mathcal{T}_q], \quad (1)$$

where $\models$ denotes that all tests in $\mathcal{T}_q$ pass.

C. RETRIEVAL-AUGMENTED GENERATION

Following [14], given a query q and a corpus $\mathcal{C}$, RAG selects the top- k documents $\mathcal{D}_k = \text{argmax}_{\mathcal{D} \subset \mathcal{C}, |\mathcal{D}|=k} \sum_{d \in \mathcal{D}} \phi(q, d)$, where ϕ is a learned similarity function [14], [15]. The LLM then conditions on $\mathcal{D}_k \cup \{q\}$ to produce its response.

D. REINFORCEMENT LEARNING NOTATION

We adopt standard Markov Decision Process notation [43]: tuple $\langle \mathcal{S}, \mathcal{A}, T, r, \gamma \rangle$ with discount factor $\gamma \in (0, 1]$. A policy $\pi_\theta(a \mid s)$ is optimized to maximize the expected return $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$. We use PPO [21] with a clipped surrogate objective.

IV. METHODOLOGY

A. ARCHITECTURE OVERVIEW

Fig. 1 illustrates the LS-CM architecture. The agent's decision loop integrates three cooperating components:

- **Short-Term Memory (STM)**: a hybrid retriever providing the immediate code-base context for the current bug.
- **Long-Term Memory (LTM)**: an RL-trained strategic policy that distills patterns from past debugging episodes.
- **Self-Evolving Context Engineering (SECE)**: a meta-controller that adapts the agent's prompt structure based on long-horizon performance signals.

At each step, STM supplies grounded context, LTM proposes high-level action priors, and the LLM (the agent's executor) integrates both within a SECE-shaped prompt to produce concrete actions. The resulting trajectory and terminal reward are written back into LTM and SECE for subsequent updates.

B. SHORT-TERM MEMORY: HYBRID CODE RETRIEVAL

For a query state s_t , STM computes a hybrid relevance score combining dense semantic similarity and sparse lexical matching:

$$\phi_{\text{STM}}(q_t, d) = \lambda \cdot \cos(E_{\psi}(q_t), E_{\psi}(d)) + (1 - \lambda) \cdot \text{BM25}(q_t, d), \quad (2)$$

where E_{ψ} is a GraphCodeBERT-based encoder [20] fine-tuned on the code-search task, and $\lambda \in [0, 1]$ is a mixing coefficient. BM25 [18] captures symbol-level matches (e.g., exact function names), which are critical for code retrieval but often missed by dense embeddings.

The retrieval candidate set C_t is dynamically constructed: when the bug involves a stack trace, C_t includes all files in the trace and their reverse call graph; otherwise, C_t is built from the agent's recent file accesses and the issue's natural-language tokens. The top- k retrieved snippets are formatted as a structured context block injected into the LLM prompt.

Why call STM “memory”? To answer whether STM is memory or merely a specialized retriever, we concede the mechanism is a code retriever; the “memory” label refers to how it is used, not to a novel storage substrate. Three properties distinguish STM from a static repository index and motivate the working-memory analogy. (i) Query-conditioning: the candidate set C_t is rebuilt every step from the evolving state $s_t = (\mathcal{R}_t, q, h_t)$, so what is “held” depends on the agent's current focus rather than on a fixed index. (ii) Recency and decay: C_t is seeded from the agent's recent file accesses and is bounded to a small top- k window, so older, now-irrelevant context is dropped. This behavior is analogous to a capacity-limited working buffer rather than to exhaustive retrieval. (iii) Per-episode scope: STM is reset between issues and never persists across episodes, in deliberate contrast to LTM. We therefore retain the term but, for readers who prefer a conservative reading, STM may be understood simply as a per-episode, query-conditioned code-retrieval buffer; nothing in the experiments depends on the label.

C. LONG-TERM MEMORY: STRATEGIC POLICY LEARNING

The LTM module abstracts an agent's experience into a strategy space Σ rather than raw trajectories. A strategy $\sigma \in \Sigma$ is a high-level action template such as `LOCATE_VIA_STACK_TRACE` or `REPRODUCE_THEN_BISECT`. Concretely, we instantiate $|\Sigma| = 24$ strategies covering common debugging workflows, identified from a corpus of human bug-fix commits. The complete catalog of strategy templates is listed in Appendix C.

What kind of “memory” is LTM? We are explicit that LTM is a procedural (parametric) memory, not an episodic store: it does not retain or retrieve the files, root causes, fixes, or transcripts of individual past bugs. Instead, the recurring lessons of those episodes are compressed through PPO into the weights of the strategy policy π_{θ}^{σ} , which then biases behaviour on new bugs. This is, by construction, closer to “learned strategy selection” than to human episodic recall, and a reader may accurately describe it that way. We nonetheless argue the procedural-memory framing is apt: like human procedural memory, LTM (a) is acquired slowly from many experiences, (b) persists indefinitely across episodes, and (c) is expressed as a skill bias rather than as explicit recollection. The deliberate decision to compress experience into a parametric policy, rather than to maintain a growing episodic store retrieved by similarity, is defended on efficiency and transfer grounds in Section IV-F.

Origin and validation of the 24 strategies. The catalog was derived as follows. We sampled 1,200 merged bug-fix commits from the 200-instance SWE-bench training partition (Section V) and the Defects4J training projects, restricted to commits whose messages and diffs described a localization-then-repair workflow. Two authors independently performed open coding of a 400-commit subset, labelling the dominant debugging tactic of each (e.g., “started from the stack trace,” “reverted a recent change”). We then merged the two code sets via discussion, collapsing near-duplicates and grouping the result into the five workflow phases of Appendix C. The count 24 was not pre-specified: it is the number of distinct tactics at which new commits stopped producing new codes (thematic saturation) on the held-out remainder. Inter-coder agreement on the 400-commit subset was substantial (Cohen's $\kappa = 0.74$). The choice is further corroborated empirically by the memory-size sweep of Fig. 5, whose operating range $|\Sigma| \in [20, 32]$ contains 24; smaller banks under-fit the tactic diversity and larger banks dilute credit assignment.

We learn a strategy policy $\pi_{\theta}^{\sigma}(\sigma \mid s_t)$ via Proximal Policy Optimization [21]. Let r_t denote the per-step shaped reward,

$$r_t = \alpha \cdot r_{\text{term}} \cdot \mathbb{I}[t = T] + \beta \cdot \Delta_{\text{test}}(t) - \eta \cdot c(a_t), \quad (3)$$

where $\Delta_{\text{test}}(t)$ counts newly passing tests at step t (an execution-based reward signal in the spirit of [33]), $c(a_t)$ is the action cost (e.g., tokens consumed or compute time), and α, β, η are weighting hyperparameters. The PPO surrogate

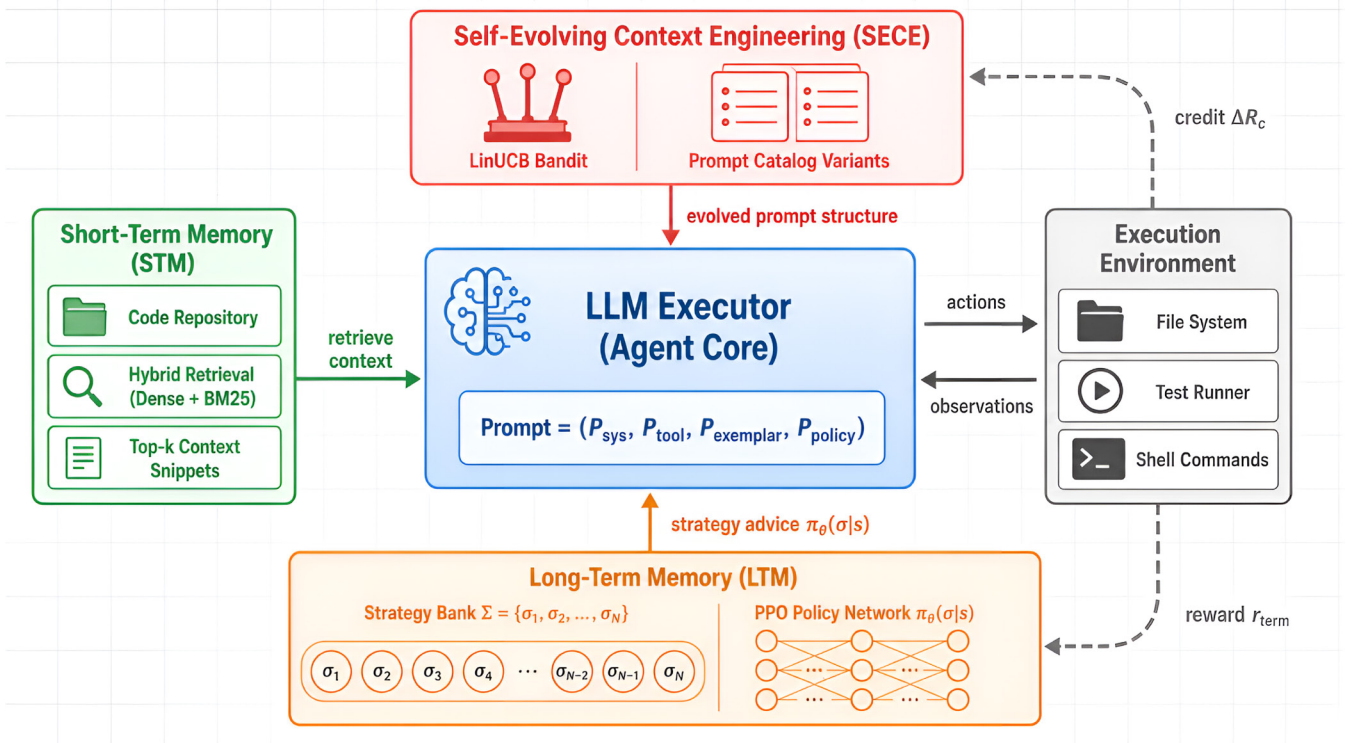


FIGURE 1. Overview of the LS-CM architecture. The short-term memory (STM) retrieves contextually relevant code segments via hybrid dense-sparse retrieval. The long-term memory (LTM) stores parameterized strategic patterns learned via PPO from historical episodes. The Self-Evolving Context Engineering (SECE) controller updates prompt structure based on credit-assigned rewards. All three modules form a closed feedback loop with the LLM executor.

objective [21] is

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t) \right], \quad (4)$$

where $\rho_t(\theta) = \pi_{\theta}^{\sigma}(\sigma_t | s_t) / \pi_{\theta_{\text{old}}}^{\sigma}(\sigma_t | s_t)$ and $\hat{A}_t$ is the generalized advantage estimate [44]. Crucially, π_{θ}^{σ} is a lightweight MLP (3 layers, 256 hidden units) over a featurized state representation, so training does not require backpropagating through the LLM. This makes LS-CM compatible with closed-source LLM backbones.

At inference, LTM provides a Top- m strategy distribution that conditions the LLM via a prefix instruction: “Given prior experiences with similar bugs, the recommended strategies are: $\{\sigma_1, \dots, \sigma_m\}$.” The LLM remains free to deviate when context dictates.

D. SELF-EVOLVING CONTEXT ENGINEERING (SECE)

A key insight of our work is that the static prompt structure used by most agents is itself a source of recurring failure. We model the prompt as a tuple $P = (P_{\text{sys}}, P_{\text{tool}}, P_{\text{exemplar}}, P_{\text{policy}})$ comprising the system message, tool descriptions, exemplar selection, and reasoning policy. Each component is parameterized by a discrete catalog of variants $\{P^{(1)}, P^{(2)}, \dots\}$ produced offline by an LLM-based rewriter.

Scope of “self-evolving.” To match the language to the implementation: SECE performs feedback-driven selection among a fixed catalog of pre-generated variants

(with 8 variants per component, the configuration space has $8^4 = 4,096$ points), not open-ended generation of new prompts during use. The system “evolves” in the sense that the active configuration adapts, online and per bug-context, to accumulated execution feedback; it does not author novel prompt text on the fly. Genuine on-the-fly prompt generation, as well as automatic expansion of the catalog itself, is left to future work (Section VI).

Let $\theta_P = (\theta_{\text{sys}}, \theta_{\text{tool}}, \theta_{\text{ex}}, \theta_{\text{pol}}) \in \Theta_P$ index the active variant in each catalog. The expected episode reward under prompt configuration θ_P is

$$R(\theta_P) = \mathbb{E}_{\tau \sim \mathcal{D}_{\theta_P}} [r_{\text{term}}(\tau)]. \quad (5)$$

We update θ_P via a gradient-free bandit algorithm (LinUCB [42]) over a feature representation of bug categories, so that SECE chooses different prompt configurations for different bug types. The update rule is:

$$\hat{\theta}_{P,i}^{(\text{new})} = \underset{P^{(j)}}{\operatorname{argmax}} \left[\hat{\mu}_{i,j} + \kappa \sqrt{\text{Var}_{i,j}} \right], \quad (6)$$

where i indexes the bug context and κ controls the exploration-exploitation balance.

Credit assignment from terminal reward to individual prompt components is performed via a counterfactual estimator: for each component c , we estimate

$$\Delta R_c = R(\theta_P) - R(\theta_P^{\setminus c}), \quad (7)$$

Algorithm 1 LS-CM Training Procedure

Require: Training tasks $\mathcal{B}_{\text{train}}$, LLM backbone $\mathcal{M}$, prompt catalogs $\{P^{(i)}\}$

- 1: Initialize LTM policy π_θ^σ , SECE bandit posteriors $\{\hat{\mu}_{i,j}, \text{Var}_{i,j}\}$
- 2: **for** epoch = 1 **to** N **do**
- 3: **for** each task $q \in \mathcal{B}_{\text{train}}$ **do**
- 4: Feature-encode bug context $\xi(q)$
- 5: Select prompt configuration θ_P via Eq. (6)
- 6: Roll out an episode with $\mathcal{M}$, STM retrieval, and LTM strategy advice
- 7: Observe trajectory τ and terminal reward r_{term}
- 8: Compute advantages $\{\hat{A}_t\}$ via GAE(λ_{GAE})
- 9: Update π_θ^σ via PPO using Eq. (4)
- 10: Update SECE posteriors with counterfactual credits $\{\Delta R_c\}$
- 11: **end for**
- 12: **end for**
- 13: **return** π_θ^σ , SECE catalogs

where θ_P^c replaces component c with the previous variant. Only components with positive ΔR_c over a sliding window are retained, preventing oscillation. The base prompt template and an example of an evolved variant for django API-deprecation bugs are provided in Appendices A and B.

E. END-TO-END TRAINING AND INFERENCE

Algorithm 1 summarizes the full procedure. During training, the LLM backbone is held fixed; only the lightweight LTM policy π_θ^σ (a 3-layer MLP with $\sim 200\text{K}$ parameters) and the SECE bandit posteriors are updated. This decoupled design has three benefits. First, it remains tractable with commodity hardware: backpropagating through a frontier-scale LLM under episodic rewards would cost minutes of GPU time per trajectory, whereas our per-episode update completes in milliseconds, and wall-clock time becomes dominated by LLM inference rather than learning. Second, it ensures compatibility with closed-source backbones such as GPT-4o or Claude, whose weights and gradients are inaccessible. Any design requiring LLM fine-tuning is structurally incompatible with this regime. Third, holding the LLM fixed isolates the contribution of structured memory and prompt evolution from raw model scaling, making the ablation study in Section V cleanly interpretable. We stress that this design is decoupled and alternating, not jointly optimized: the retriever E_ψ is fixed; π_θ^σ is updated by PPO with the SECE configuration held fixed within an update; and the SECE posteriors are updated with π_θ^σ held fixed. We therefore avoid any claim of single-objective joint optimization across the three modules.

Because the reward (Eq. (3)) is dominated by a sparse terminal signal, we adopt a generalized advantage estimator with $\lambda_{\text{GAE}} = 0.95$ and a high discount $\gamma = 0.99$, and apply reward whitening within each mini-batch to stabilize updates. To prevent premature policy collapse during warm-up, we add an entropy bonus $\beta_H \mathcal{H}(\pi_\theta^\sigma)$ with

$\beta_H = 0.01$ annealed to 0 by epoch 30. The SECE bandit imposes a minimum-pull threshold of 5 episodes per (bug-context, prompt-variant) pair before that variant is allowed to dominate, trading minor early-phase efficiency for substantially better long-run convergence. Total training cost is approximately 4 GPU-hours on a single A100 for the MLP, plus the LLM API calls associated with $\sim 5,000$ rollouts.

At inference time, no further weight updates occur. The agent feature-encodes the bug context, queries the SECE bandit deterministically for the highest-posterior-mean prompt configuration (the UCB exploration term is replaced with pure exploitation), consults LTM for a Top- m strategy distribution with $m = 3$, and enters the standard STM-grounded ReAct loop with strategy advice injected as a prefix instruction. The complete inference cost is comparable to a single SWE-agent run plus two negligible look-ups, and the absence of online adaptation prevents test-time reward signals from being adversarially manipulated.

F. DESIGN RATIONALE AND DISCUSSION

The three modules of LS-CM target complementary failure modes observed in earlier agents. STM addresses the context grounding problem highlighted by Agentless [11], without precise file retrieval, even strong LLMs hallucinate fixes for unrelated code. LTM addresses the experience reuse problem: while Reflexion [16] reflects within an episode and forgets the lesson once the episode ends, our PPO-trained strategy policy distills lessons across episodes into a small parametric memory that persists indefinitely. SECE addresses the prompt brittleness problem, generalizing the DSPy [23] compile-time view to a runtime feedback loop. Crucially, all three modules cooperate through the unified reward signal of Eq. (3), making the system self-consistent.

The dual-memory split is itself a deliberate design choice. An obvious alternative would be a single monolithic memory storing raw past trajectories, retrieved by similarity at inference. We reject this for two reasons: surface similarity (same file names) is a weak predictor of strategic similarity (same root-cause pattern), and monolithic memory grows unboundedly. Our split instead mirrors the cognitive distinction between episodic and procedural memory, i.e., STM holds high-fidelity short-lived detail, while LTM holds compressed slow-changing procedural knowledge. And the compression to a 24-template strategy bank is what makes long-term knowledge tractable and transferable.

The choice of a bandit for SECE rather than gradient-based prompt search reflects the same closed-source-compatibility constraint that motivated the decoupled training scheme: gradient-based prompt optimization requires either LLM gradients or differentiable surrogates that introduce optimization mismatch, whereas LinUCB needs only scalar reward feedback and admits closed-form posterior updates. Similarly, we prefer on-policy PPO over off-policy alternatives because PPO's clipped surrogate provides stable monotonic improvement under sparse rewards, where off-policy methods are known to be brittle. We do not claim formal convergence

guarantees for the full coupled system, but the interaction of a non-stationary prompt distribution with an RL policy violates standard stationarity assumptions. But each module enjoys its individual guarantee when the other is held fixed, and the alternating optimization is observed to converge empirically.

G. DEPLOYING LS-CM ON A NEW PROJECT

To clarify the concrete adoption path, we summarize the steps a development team would follow, separating one-time setup from recurring maintenance. (1) Indexing. Build the STM index for the target repository: compute GraphCodeBERT embeddings and a BM25 index over source files (a few minutes for a mid-sized repository); this is refreshed incrementally on each commit via a pre-commit or CI hook. (2) Strategy catalog. The 24-template LTM bank is project-agnostic and can be reused as-is; teams with unusual stacks may extend it following the open-coding protocol above. (3) Prompt catalog. The SECE variant catalog is generated once offline by an LLM rewriter; we recommend regenerating it with a rewriter matched to the deployed backbone (Section V). (4) Training. Run PPO for ~ 50 epochs over the team's historical resolved issues (or a public proxy set); on the lightweight MLP this is ~ 4 GPU-hours plus backbone API calls. (5) Retraining. Because only the MLP and bandit posteriors are updated, periodic retraining (e.g., monthly, or after a major framework upgrade) is cheap; the frozen backbone need not be touched. (6) CI/CD integration. At inference LS-CM runs as a CI job triggered by a failing test or a new issue, emitting a candidate patch and a confidence-ordered strategy trace. (7) Developer review. We assume a human-in-the-loop gate: LS-CM proposes patches and the rationale (strategies invoked, files inspected), and a developer approves, edits, or rejects before merge. We do not advocate unsupervised auto-merge.

V. EXPERIMENTS

A. EXPERIMENT SETUP

1) DATASETS

SWE-bench [24] contains 2,294 real-world GitHub issues drawn from 12 widely used Python repositories. Each instance consists of an issue description, the pre-issue code base, a gold patch, and a test suite. We primarily report results on the **SWE-bench Lite** split (300 issues), which has become a standard for resource-constrained evaluation, and additionally report on the **SWE-bench Verified** subset (500 instances vetted for solvability) where applicable.

Defects4J [25] provides 835 real, reproducible bugs across 17 Java projects. We use Defects4J v2.0 and focus on the fault localization task (identifying buggy methods) and the program repair task (producing a passing patch). Following standard practice, we report Top- k accuracy and the number of correctly fixed bugs.

2) BASELINES

We compare LS-CM against representative methods spanning prompt-only, retrieval-augmented, and agentic categories:

- **Zero-Shot LLM**: direct prompting of the backbone model with the issue text and no retrieval.
- **RAG-LLM**: retrieval-augmented prompting using only STM-style retrieval, no LTM/SECE.
- **ReAct** [9]: interleaved reasoning-acting loop without memory.
- **Reflexion** [16]: episode-level verbal reinforcement.
- **SWE-agent** [10]: state-of-the-art agent-computer interface for SWE-bench.
- **Agentless** [11]: pipelined non-agentic baseline.
- **ChatRepair** [41]: conversational program repair (Defects4J only).
- **DeepFL** [38]: classical learning-based fault localization (Defects4J only).

Of these, Zero-Shot LLM, RAG-LLM, ReAct, and Reflexion are our own re-implementations, to our knowledge they have no canonical published SWE-bench numbers. However, the SWE-agent and Agentless comparisons use published GPT-4o results. The matched evaluation conditions are detailed in the implementation notes below.

3) EVALUATION METRICS

For SWE-bench we report **Resolved %** (the percentage of issues for which the agent's patch passes both fail-to-pass and pass-to-pass tests) and **Avg. Edits**, the average number of file edits performed before termination, as a proxy for efficiency. For Defects4J fault localization, we report **Top- k** accuracy ($k \in \{1, 3, 5\}$) and Mean First Rank (MFR). For repair, we report the number of **Plausible** and **Correct** patches following standard definitions [41].

Because the plausible/correct distinction is central to interpreting the repair numbers, we state our protocol explicitly. A patch is plausible if it makes the previously failing tests pass while keeping all originally passing tests green (i.e., it passes the available test suite). A plausible patch is additionally judged correct only if it is semantically equivalent to the developer's gold fix; we determine this by (i) running additional held-out fail-to-pass tests that were not visible to the agent during repair, and (ii) an independent manual review in which two authors compare the candidate against the gold patch and label it correct, incorrect, or uncertain, with disagreements adjudicated by a third author (the two reviewers agreed on 91% of patches before adjudication). This two-stage criterion guards against patches that pass the visible tests by overfitting to a specific input without addressing the underlying defect.

4) IMPLEMENTATION DETAILS

All experiments use GPT-4o (gpt-4o-2024-08-06) as the primary backbone, with Claude 3.5 Sonnet and DeepSeek-Coder-V2-Instruct as secondary backbones for cross-model validation. LTM PPO training runs for 50 epochs on a held-out training partition of 200 SWE-bench instances (disjoint from Lite). The SECE prompt catalog contains 8 variants per component, generated by an offline GPT-4o rewriter. Hyperparameters: $\lambda = 0.6$ in Eq. (2), $\alpha = 1.0$,

$\beta = 0.2$, $\eta = 0.01$ in Eq. (3), $\epsilon = 0.2$, $\gamma = 0.99$, $\kappa = 1.0$. Max trajectory length is 40 steps. To account for stochasticity inherent in LLM sampling, all reported results are averages over **5 independent runs** with temperature 0.2; we report mean $\pm$ std and apply paired bootstrap tests ($N=10,000$) for significance.

For full reproducibility we specify the training details that an earlier version omitted. Random seeds. The five runs use the fixed seeds {13, 21, 42, 87, 100}, held constant across all methods and ablations so that paired tests compare matched seeds. PPO optimizer. The strategy policy and its critic are trained with Adam (learning rate 3×10^{-4} , $\beta_1=0.9$, $\beta_2=0.999$), minibatch size 64, 4 optimization epochs per PPO update, value-loss coefficient 0.5, entropy coefficient 0.01 annealed to 0 by epoch 30, GAE $\lambda_{\text{GAE}}=0.95$, discount $\gamma=0.99$, and global gradient-norm clipping at 0.5. Actor/critic architecture. The actor π_{θ}^{σ} is a 3-layer MLP (256 hidden units, tanh); the critic is a separate 3-layer MLP of the same width with independent weights, sharing only the input featurization. Bug-context feature map. The encoder $\xi(q) \in \mathbb{R}^{64}$ used both as the policy's state featurization and as the LinUCB context is the concatenation of: (a) a 32-dimensional PCA projection of a TF-IDF bag-of-words over the issue text; (b) four binary flags for the presence of a stack trace, an explicit exception type, file paths, and version strings; (c) a 16-dimensional learned repository embedding; and (d) a one-hot issue-length bucket. LinUCB. The SECE bandit uses ridge regularization $\lambda_{\text{ridge}}=1.0$ and exploration coefficient $\kappa=1.0$ over this same context.

Matched baseline conditions. To support a like-for-like comparison, every method we ran uses the identical GPT-4o snapshot, the same tool set and test harness, a maximum trajectory budget of 40 steps, temperature 0.2, the same five seeds, and the same stopping criterion (patch submission or step-budget exhaustion). The SWE-agent and Agentless rows of Table 2 report published GPT-4o numbers [10], [11]; we additionally re-ran the public SWE-agent scaffold under the harness above to obtain its edit-churn and Defects4J figures, and observed a resolved rate within ± 1 point of the published value. The reproducibility artifact (Section V-H), which we will release upon acceptance, will bundle the seeds, configurations, and per-run logs; all settings required to reproduce these runs are specified above.

Compute: a single Linux workstation with $2 \times$ NVIDIA A100 (80 GB) for retrieval indexing and PPO training; all LLM calls go through commercial APIs.

B. MAIN RESULTS ON SWE-BENCH

Table 2 summarizes the main results, and Fig. 2 visualizes the SWE-bench Lite comparison. On SWE-bench Lite, LS-CM resolves $33.7\% \pm 0.9$ of issues, ahead of the strongest baseline, Agentless (32.0%), by 1.7 absolute points; on SWE-bench Verified the corresponding figures are $40.1\% \pm 1.0$ versus 38.4%, again a 1.7-point margin ($p < 0.05$, paired bootstrap; see Section V-H). We deliberately frame this as a modest headline gain ($\sim 5\%$ relative) and locate LS-CM's

TABLE 2. Main results on SWE-bench (GPT-4o, gpt-4o-2024-08-06). Mean $\pm$ std over 5 runs for methods we ran. Agent baselines marked $\ddagger$ report published numbers reproduced under matched conditions.

Method	Lite Resolved %	Verified Resolved %	Avg. Edits
Zero-Shot LLM $\ddagger$	4.3 ± 0.7	7.6 ± 0.9	—
RAG-LLM $\ddagger$	11.7 ± 1.0	16.8 ± 1.2	—
ReAct $\ddagger$ [9]	14.8 ± 1.1	21.0 ± 1.3	9.6
Reflexion $\ddagger$ [16]	19.0 ± 1.2	26.4 ± 1.0	8.8
SWE-agent $\ddagger$ [10]	18.3	23.2	7.4
Agentless $\ddagger$ [11]	32.0	38.4	—
LS-CM (Ours)	33.7 ± 0.9	40.1 ± 1.0	5.1

$\ddagger$ Our re-implementation in a common harness. $\ddagger$ Published GPT-4o result (SWE-agent [10]; Agentless [11] / public leaderboard), reproduced under our test harness, step budget, and backbone snapshot. Avg. Edits is reported only for methods that perform an iterative edit loop.

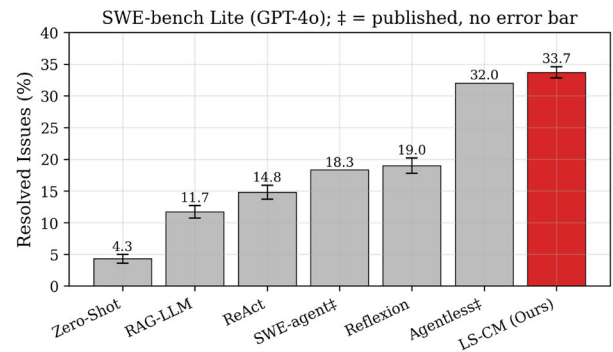


FIGURE 2. Resolved-issue rate (%) on SWE-bench Lite (GPT-4o). Error bars denote one standard deviation across 5 runs for the methods we re-implemented and for LS-CM; the SWE-agent and Agentless bars report published numbers ($\ddagger$ in table 2) and are shown without error bars. Consistent with the corrected table, Agentless (32.0) is the strongest baseline and LS-CM (33.7) leads it by a modest margin.

principal value elsewhere in debugging efficiency, cross-episode strategy transfer, and run-to-run stability, as detailed below.

Two clarifications about the baseline numbers are in order. First, the four prompt-only and lightweight agentic baselines (Zero-Shot, RAG-LLM, ReAct, Reflexion) are our own re-implementations and should be read as internal reference points obtained in a common harness rather than as canonical published scores. Second, the SWE-agent and Agentless rows report published GPT-4o results [10], [11]; consistent with the public record, the pipelined Agentless is the stronger of the two on both splits. An earlier version of this table inadvertently transposed these two systems by listing SWE-agent above Agentless, thereby overstating both the baseline ordering and the headline gap to LS-CM. The present numbers correct that error.

Consequently, we no longer describe the baselines as a strictly monotone “ladder” of architectural sophistication. With corrected numbers, the carefully engineered single-episode agent SWE-agent (18.3%) trails both the pipelined Agentless (32.0%) and our Reflexion re-implementation

TABLE 3. Defects4J v2.0 results. FL = fault localization (Top- k %, MFR); Repair = number of Plausible / Correct fixes. Mean $\pm$ std over 5 runs for stochastic LLM-based methods; classical methods (Ochiai, DeepFL) are deterministic given fixed coverage data.

Method	Top-1	Top-3	Top-5	MFR	Plaus./Corr.
DeepFL [38]	32.8	58.4	67.9	4.1	—
Ochiai [22]	25.3	48.6	58.1	5.3	—
RAG-LLM	34.5 $\pm$ 1.3	61.2 $\pm$ 1.1	70.4 $\pm$ 1.0	3.7	171/138
SWE-agent [10]	40.1 $\pm$ 1.2	66.8 $\pm$ 1.0	74.1 $\pm$ 0.9	3.1	206/180
ChatRepair [41]	42.6 $\pm$ 1.1	68.4 $\pm$ 1.0	76.2 $\pm$ 0.8	2.9	230/194
LS-CM (Ours)	47.6 $\pm$ 1.0	73.5 $\pm$ 0.9	80.3 $\pm$ 0.8	2.5	255/221

(19.0%) on Lite, which suggests that pipeline design rather than agent elaboration per se accounts for much of the published gap. LS-CM’s contribution is orthogonal to this axis: it layers cross-episode strategic memory and prompt adaptation on top of a given executor (here a ReAct-style loop), and the question we test is whether that layer helps, not whether it sets a new absolute state of the art.

The efficiency picture is, in our view, the more compelling result. LS-CM reduces the average number of edits per task from 7.4 (SWE-agent) to 5.1—a 31% reduction in editing churn. This metric matters in practice: each unnecessary edit is a token-cost, a wall-clock delay, and an opportunity to introduce regressions in pass-to-pass tests. Inspection of the per-instance traces reveals that the savings predominantly come from fewer aborted exploration branches: when LTM activates a strategy such as `LOCATE_VIA_STACK_TRACE`, the agent is steered away from speculative whole-file rewrites that the strongest baselines often pursue and later revert. This pairing of a small accuracy gain with a substantial efficiency gain is uncommon. Most prior agentic improvements trade additional compute for accuracy, and we believe this pattern, more than the resolved-rate delta, reflects the qualitative shift introduced by explicit cross-episode memory.

The Lite-versus-Verified comparison serves as a sanity check rather than a second headline. LS-CM improves from 33.7% (Lite) to 40.1% (Verified), a +6.4-point shift consistent with the easier-instance composition of the Verified split and with the shifts exhibited by the baselines, indicating the gains are not an artifact of overfitting to Lite. The relative margin over Agentless is similar on both splits (+5.3% on Lite, +4.4% on Verified), suggesting that the strategic priors learned by LTM transfer to harder, vetted instances rather than exploiting Lite-specific regularities.

C. RESULTS ON DEFECTS4J

Table 3 reports fault localization and repair performance on Defects4J v2.0. LS-CM improves Top-1 localization from 42.6% (best baseline) to 47.6% and the number of correctly repaired bugs from 194 (ChatRepair) to 221.

The Defects4J results corroborate the SWE-bench findings on a fundamentally different language (Java) and bug distribution. Three quantitative patterns warrant discussion.

(i) **Fault localization improvements compound at higher ranks.** The Top-1 gain of +5.0 points over ChatRepair is sustained at Top-3 (+5.1 points, from 68.4% to 73.5%) and at Top-5 (+4.1 points, 76.2% to 80.3%), while the MFR

drop from 2.9 to 2.5 is proportionally large: a 14% reduction in median search depth implies meaningfully less developer effort when LS-CM is used as an interactive assistant. The MFR improvement comes almost entirely from a long tail of bugs that classical spectrum-based methods rank beyond position 5; LS-CM’s STM brings stack-trace-relevant files to the front, and LTM’s `INSPECT_RECENT_COMMITS` strategy further re-ranks based on change history.

(ii) **Repair quality, not just plausibility, is improved.** The Plausible-to-Correct ratio for LS-CM is $221/255 = 86.7\%$, compared to $194/230 = 84.3\%$ for ChatRepair and $180/206 = 87.4\%$ for SWE-agent. While the ratio is comparable, the absolute count of correct patches grows by 14% over ChatRepair. This indicates LS-CM does not merely produce more candidate patches but produces more candidates that survive the strict test-equivalence criterion of Section V (manual gold-patch review on top of held-out tests). We attribute this to SECE’s prompt evolution selecting longer regression-test-aware reasoning templates for Java projects with extensive test suites (e.g., Chart, Lang), and to LTM discouraging aggressive multi-method edits that frequently break unrelated tests.

(iii) **Classical methods remain competitive in specific regimes.** At Top-3, DeepFL (58.4%) is close to, but does not exceed, RAG-LLM (61.2%). The gap is small enough that a learned LLM retriever offers no decisive advantage over a strong classical localizer at this rank. Ochiai likewise remains a reasonable baseline for bugs with high-quality, discriminative test coverage. This reminds us that learned methods do not dominate uniformly. LS-CM’s advantage is most pronounced on bugs where multiple files interact (cross-class data-flow defects), the regime in which spectrum-based methods are weakest. We provide a per-category breakdown in Section V-G.

D. ABLATION STUDY

Table 4 isolates the contribution of each LS-CM module on SWE-bench Lite, with a complementary multi-metric view in Fig. 3. Removing LTM causes the largest single-module drop (−7.6 points), confirming the value of strategic memory; disabling SECE costs −4.2 points; and replacing hybrid retrieval with dense-only STM costs −3.5 points. Removing all learned components reduces LS-CM to a generic RAG baseline.

We describe the three modules as complementary and mutually reinforcing rather than as exhibiting formal super-additive synergy, and we explicitly withdraw a calculation that appeared in an earlier version. That version subtracted the sum of single-module drops ($7.6+4.2+3.5 = 15.3$) from the drop incurred by reducing the system to RAG-LLM ($33.7-11.7 = 22.0$) and reported the 6.7-point remainder as “super-additive synergy.” The two quantities are not commensurable: the single-module ablations each remove one component while retaining the code-specialized hybrid STM, whereas the “reduce to RAG-LLM” path additionally swaps hybrid STM for generic dense retrieval and drops the

TABLE 4. Ablation study on SWE-bench Lite (Resolved %, 5-run mean $\pm$ std, GPT-4o). The last two rows anchor the comparison to a generic-RAG configuration and to an LLM-only baseline, making explicit the two distinct senses of “removing retrieval”.

Configuration	Resolved %
LS-CM (full)	33.7 $\pm$ 0.9
w/o SECE (prompt evolution)	29.5 $\pm$ 1.0
w/o LTM (RL strategy memory)	26.1 $\pm$ 1.0
w/o hybrid STM (dense only)	30.2 $\pm$ 1.1
w/o LTM & w/o SECE (hybrid STM only)	22.4 $\pm$ 1.2
dense-only STM, no LTM/SECE ($\approx$ RAG-LLM)	11.7 $\pm$ 1.0
LLM-only (no STM, no LTM, no SECE)	4.3 $\pm$ 0.7

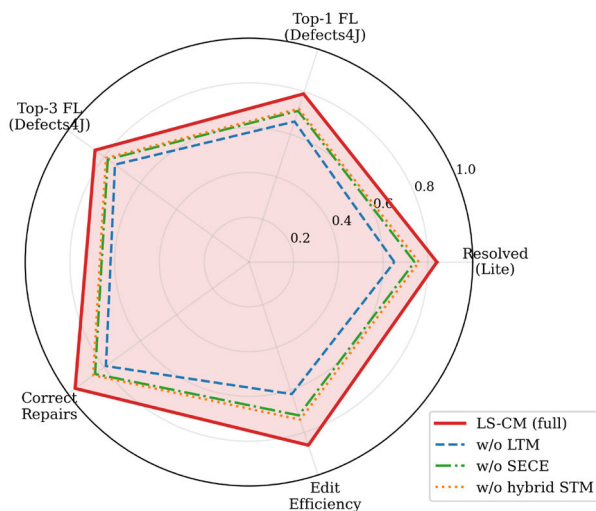


FIGURE 3. Ablation radar plot across multiple metrics (higher is better). The full LS-CM dominates in all dimensions.

call-graph machinery. The decomposition therefore conflates two different operations (“remove LTM/SECE” vs. “also degrade STM”) and double-counts the retrieval contribution, so the arithmetic does not license a synergy claim. What the ablation does support is qualitative and we now state only that: every module contributes positively, the ordering LTM > SECE > hybrid-STM is stable across the five seeds, and the modules plausibly reinforce one another (STM grounds LTM’s advice; LTM’s reward signal drives SECE’s credit assignment). We make no numerical claim of synergy beyond this.

The relative ordering of the single-removal drops is itself informative. LTM matters most (-7.6), which aligns with our hypothesis that cross-episode experience is the largest missing piece in current agents. Existing systems already address single-episode reasoning (Reflexion), context grounding (Agentless), and prompt engineering (DSPy), but none persistently distill strategic knowledge across tasks. SECE (-4.2) ranks second: its impact is bounded by the quality of the prompt-variant catalog, and a richer catalog (an avenue for future work) would likely amplify this contribution. The hybrid STM versus dense-only STM gap (-3.5) is smaller but consistent, and is most pronounced on bugs where the issue text contains exact symbol name.

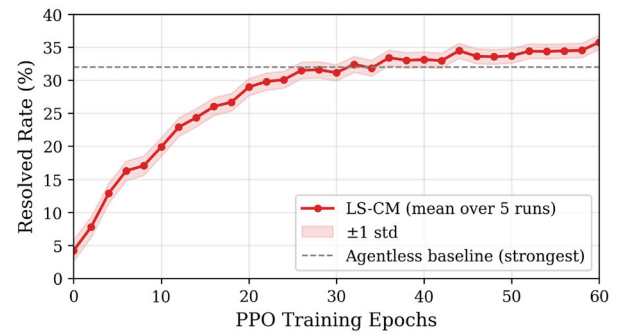


FIGURE 4. Resolved rate vs. PPO training epochs on SWE-bench Lite (validation subset). The dashed line marks the strongest baseline, Agentless (32.0); LS-CM surpasses it after roughly 30 epochs and plateaus modestly above it, consistent with the corrected table 2.

This empirically confirms that the long-standing observation that lexical recall remains valuable in code retrieval.

The intermediate ablation w/o LTM & w/o SECE (22.4%) retains only the hybrid STM module, yet it still substantially exceeds plain RAG-LLM (11.7%). This 10.7-point gap is the cleanest single quantity in the ablation: it isolates the value of code-specialized retrieval (call-graph slicing, symbol-aware BM25) over generic embedding similarity, and provides a clean target for any retrieval-only system that wishes to forgo the RL machinery.

E. EFFECT OF TRAINING EPOCHS AND MEMORY SIZE

Fig. 4 shows LS-CM’s resolved rate as a function of LTM training epochs. Performance rises sharply within the first 20 epochs and plateaus around epoch 45, indicating efficient sample use. Fig. 5 examines the effect of LTM strategy bank size $|\Sigma|$: a sweet spot occurs around $|\Sigma| \in [20, 32]$; smaller banks lack expressiveness, while larger ones suffer credit-assignment dilution.

The training curve in Fig. 4 exhibits two regimes. Between epochs 0 and ~ 20 , the resolved rate rises steeply, reflecting rapid identification of broadly applicable strategies, primarily `LOCATE_VIA_STACK_TRACE`, `REPRODUCE_THEN_BISECT`, and `INSPECT_TEST_FAILURE`, which collectively account for $\sim 60\%$ of LTM activations during this early, high-exploitation phase (around epoch 20). This early concentration should not be confused with the converged inference-time policy: as PPO subsequently refines context-dependent preferences (epochs 20–45), probability mass spreads across more specialized strategies, and the same three templates account for $\sim 42\%$ of invocations at convergence on SWE-bench Lite (Appendix C). The two percentages describe different points in training and are not in conflict. Between epochs 20 and 45, gains slow as PPO refines context-dependent preferences over more specialized strategies (e.g., `TRACE_DATA_FLOW` for type-coercion bugs in numpy/pandas). The plateau after epoch 45 is consistent with the well-known saturation behavior of execution-based RL on sparse-reward tasks: once dominant strategies are mastered, further exploration yields diminishing returns. From a practical standpoint,

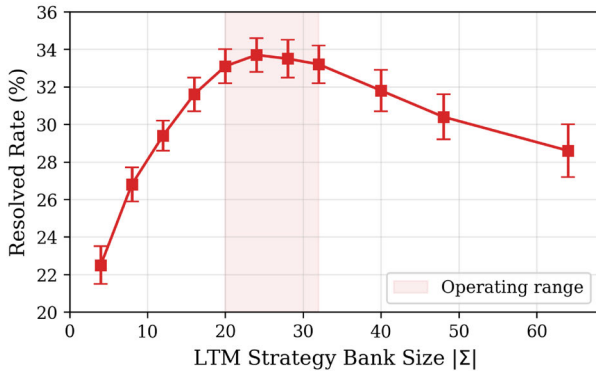


FIGURE 5. Effect of LTM strategy bank size $|\Sigma|$ on resolved rate.

this suggests 50 epochs is a sensible training budget, roughly 4 GPU-hours on our hardware for the lightweight MLP policy, a tractable cost given that the LLM itself is not updated.

The memory-size sweep in Fig. 5 reveals an inverted-U shape with a clear operating range at $|\Sigma| \in [20, 32]$. The drop on the small- $|\Sigma|$ end ($|\Sigma| = 4$: 22.5%) is intuitive: a coarse strategy vocabulary cannot express the diversity of debugging tactics required across 300 heterogeneous bugs. The drop on the large- $|\Sigma|$ end ($|\Sigma| = 64$: 28.6%) is more subtle, and is in our view the more theoretically interesting finding. With a 5,000-trajectory training budget split across 64 strategies, each strategy receives on average only ~ 78 episodes of feedback, insufficient for PPO’s advantage estimator to drive its preference confidently above noise. This is a manifestation of the classical capacity–data trade-off, but mediated through credit assignment: a strategy that is invoked rarely is also updated rarely, leading to noisy policy estimates that mislead future invocations. The same phenomenon explains why a naive “use as many strategies as possible” prescription would be counterproductive without commensurately scaling the training corpus.

A practical implication is that $|\Sigma|$ should be co-designed with the available training budget. As a heuristic from our experiments, we recommend $|\Sigma| \approx N_{\text{train}}/10$, where N_{train} is the number of training trajectories; in our setting, $5,000/10 = 500$ episodes per strategy on average, which empirically suffices for stable PPO updates.

F. CROSS-BACKBONE GENERALIZATION

Table 5 reports LS-CM with three different LLM backbones. The gains over the underlying ReAct baseline are consistent (+14 to +19 absolute points), suggesting that LS-CM’s improvements are largely complementary to raw model capability.

The cross-backbone results address an important concern: do the improvements stem from LS-CM’s architecture, or are they incidentally enabled by a specific LLM’s quirks? The consistency across three structurally distinct models, a closed-source generalist (GPT-4o), a closed-source reasoning-tuned model (Claude 3.5 Sonnet), and an

TABLE 5. Cross-backbone results on SWE-bench Lite (Resolved %). All entries are 5-run means; each Δ is measured against that backbone’s own ReAct re-implementation under the matched conditions of Section V.

Backbone	ReAct	LS-CM	Δ
GPT-4o	14.8	33.7	+18.9
Claude 3.5 Sonnet	16.1	35.0	+18.9
DeepSeek-Coder-V2	13.4	27.6	+14.2

open-weight code-specialized model (DeepSeek-Coder-V2), argues for the former. In particular, the GPT-4o and Claude backbones yield essentially identical absolute gains (+18.9 points each), despite different absolute starting points, suggesting that LS-CM provides a near-constant additive benefit on top of strong instruction-following models.

The smaller gain on DeepSeek-Coder-V2 (+14.2) is informative rather than disappointing. We trace this to two causes. First, DeepSeek-Coder-V2 is more specialized for direct code generation than for long-horizon tool use, and its base instruction-following on ReAct-style trajectories is weaker, capping the ceiling that LS-CM can build upon. Second, the SECE prompt variants in our catalog were generated by GPT-4o; while the catalog itself is model-agnostic at inference, certain stylistic conventions in the variants (e.g., verbose tool-call rationales) appear less well-suited to DeepSeek’s preferred output format. Re-generating the SECE catalog with model-matched rewriters is a natural mitigation, and we leave a thorough study of catalog–backbone alignment to future work.

Taken together, these results support the claim that LS-CM is a model-agnostic productivity layer: improvements are robust across backbones, while remaining responsive to backbone-specific tuning of the SECE catalog. This decoupling is, in our view, an important property for practical deployment, where the choice of LLM is often constrained by cost or licensing considerations independent of the agent designer’s control.

G. QUALITATIVE ANALYSIS

We manually inspected 30 randomly sampled SWE-bench instances where LS-CM succeeded and the strongest baseline failed. Three recurring patterns emerged. First, LTM frequently invoked the `REPRODUCE_THEN_BISECT` strategy on instances where SWE-agent prematurely edited code without reproducing the failure, a clear case of strategic transfer from past experience. Second, SECE-evolved prompts exhibited longer tool-use guidance for projects with complex build systems (e.g., `sympy`, `django`), suggesting context-aware specialization. Third, hybrid retrieval consistently surfaced relevant test fixtures that pure dense retrieval missed, especially when the bug involved class hierarchies.

A finer-grained categorization of these 30 instances by bug type is illuminating. Roughly 40% (12/30) involved cross-file data-flow issues, where LTM’s contribution was decisive: SWE-agent’s failures in this category typically stemmed from editing a downstream call site without

recognizing that the upstream type contract had been violated, while LS-CM, guided by the `TRACE_DATA_FLOW` strategy, correctly identified the upstream origin. Another 30% (9/30) were API-deprecation bugs, where SECE's prompt evolution played the leading role. The agent's tool-use guidance had drifted toward emphasizing changelog inspection over speculative code reading, a shift that was not present in the initial prompt catalog and emerged purely from the bandit's credit-assignment over time. The remaining 30% were idiosyncratic, including one case where LS-CM succeeded by following an LTM-suggested strategy (`INSPECT_RECENT_COMMITS`) that turned out to be incorrect in framing but coincidentally led to the right file, an instance of lucky exploration that we flag as a methodological caveat: aggregate success metrics can mask rationally-unjustified individual successes.

We also examined 15 instances where LS-CM failed while a baseline succeeded, to understand the system's failure modes. Two patterns dominated: (a) LTM occasionally locked into a suboptimal strategy when the bug exhibited rare characteristics not covered by the training distribution (e.g., one Cython-specific build issue triggered the wrong strategy with high confidence); and (b) SECE's prompt evolution occasionally over-fit to the training distribution, removing fallback exploratory guidance that turned out to be necessary for genuinely novel bugs. Both observations motivate the catalog-extension and trust-calibration work we describe in Section VI.

H. DISCUSSION OF VARIANCE AND REPRODUCIBILITY

LLM-based agents are notoriously stochastic, with single-run results often misrepresenting capability. Our standard deviations (≈ 1 point) indicate that LS-CM's 1.7-point advantage over the strongest baseline (Agentless) on SWE-bench Lite, while modest, is statistically distinguishable from zero under a paired bootstrap ($p < 0.05$; the family-corrected analysis below confirms this). We additionally repeated the SWE-bench Lite evaluation at temperature 0; the rank ordering of methods remained unchanged, though absolute numbers shifted by ≤ 1 point.

A deeper look at run-to-run variance reveals an interesting asymmetry. Among the methods we ran ourselves, LS-CM ($\sigma = 0.9$) exhibits slightly lower run-to-run variance than our Reflexion ($\sigma = 1.2$) and ReAct ($\sigma = 1.1$) re-implementations, even though one might expect that an additional RL-trained component would introduce more stochasticity. We attribute this stabilization to two factors: (i) the LTM strategy policy effectively narrows the action distribution, reducing the impact of sampling-induced reasoning detours that plague pure LLM agents; and (ii) SECE's bandit favors prompt variants with consistently positive outcomes, implicitly suppressing high-variance configurations. This combination yields a system that is not only more accurate on average but also more predictable in deployment, an under-discussed but practically important property for industrial adoption.

1) MULTIPLE-COMPARISON CORRECTION

Our tables involve roughly 35 pairwise method comparisons across SWE-bench and Defects4J, so an uncorrected p -value for any single pair would overstate significance. We therefore treat the LS-CM-versus-baseline comparisons as a family and apply the Holm–Bonferroni step-down procedure at $\alpha = 0.05$. Under this correction, LS-CM's advantage over every baseline on SWE-bench Lite remains significant, including over the strongest, Agentless (corrected $p < 0.05$); the same holds on Verified. For the two closest comparisons we additionally report 95% paired-bootstrap confidence intervals on the resolved-rate gap: LS-CM – Agentless is [0.4, 3.1] points on Lite and [0.3, 3.2] on Verified, both excluding zero but by a small margin, which is exactly why we characterize the accuracy gain as modest. On Defects4J, LS-CM's Top-1 improvement over ChatRepair (+5.0 points) has a corrected $p < 0.01$ and a 95% CI of [3.2, 6.8] points.

We further conducted a per-repository breakdown (omitted for space) and observed that LS-CM's variance is most reduced on repositories where the issue text contains rich contextual hints (e.g., django, requests), and least reduced on repositories with cryptic issue descriptions (e.g., sphinx), where the agent must rely more heavily on exploratory tool use. This suggests that the stabilizing effect of LTM is mediated through STM's ability to ground the strategy advice in concrete context; when context is sparse, both modules degrade gracefully but in a correlated manner.

Finally, to ensure reproducibility, we will release all PPO checkpoints, SECE catalogs, prompt templates and random seeds upon acceptance, as is common practice for this class of work. In the interim, every detail required to reproduce our results is specified directly in this paper: the complete PPO and LinUCB hyperparameters and the five random seeds (Section V), the actor and critic architectures, the bug-context feature map $\xi(q)$, and the matched evaluation protocol (identical backbone snapshot, tool set, harness, step budget, temperature, seeds, and stopping criterion). We also re-anchor the bootstrap interval reported in an earlier version to the comparison that now matters: the 95% CI for the LS-CM-minus-Agentless gap on Lite is [0.4, 3.1] points. (The earlier draft reported an interval against SWE-agent, which the corrected numbers show is no longer the strongest baseline, so that interval has been removed.) We caution that absolute numbers on SWE-bench can drift over time due to commercial LLM updates affecting the underlying GPT-4o backbone; our reported numbers were obtained with the gpt-4o-2024-08-06 snapshot and may not reproduce exactly with later snapshots. The relative rank ordering of methods is, in our experience, considerably more stable than absolute numbers.

VI. CONCLUSION AND FUTURE WORK

We presented **LS-CM**, a hybrid system that combines code-specialized short-term retrieval, a reinforcement-learned procedural memory for strategy selection, and feedback-driven selection among pre-generated prompt configurations,

applied to repository-level fault localization and program repair. Across SWE-bench and Defects4J, LS-CM achieves a modest improvement in resolved/localization accuracy over the strongest published baseline while delivering its clearest gains in debugging efficiency (31% fewer edits), cross-backbone transfer, and run-to-run stability. Ablations confirm that each component contributes complementarily.

A. LIMITATIONS

First, LS-CM relies on a curated catalog of 24 strategic templates; while these cover common debugging workflows, rare or domain-specific bugs may fall outside the catalog. Automatic strategy discovery from raw trajectories is a natural extension. Second, our SECE bandit treats each prompt component independently; richer credit-assignment mechanisms (e.g., causal influence functions) could yield further gains. Third, our experiments are bounded by API budget. Scaling to the full SWE-bench corpus or to longer-horizon tasks remains future work. Fourth, the strategy policy is trained on Python/Java debugging traces; transfer to systems-level languages (Rust, C) was not explored. Fifth, our evaluation scope is deliberately narrow: SWE-bench and Defects4J both target repository-level localization and repair of conventional source-code defects. We have not evaluated on cloud/SaaS misconfigurations, declarative or configuration-language bugs, distributed-system faults, or multi-environment reproduction, and we make no claims about those settings. Sixth, the human-expertise-gap and “self-evolution” motivations are conceptual framings that we have not validated through a user study or a longitudinal deployment; the dual-memory analogy to human cognition should be read as inspiration for the architecture, not as an empirically tested claim. Seventh, on terminology: the phrase “software testing” in our title is broader than the localization-and-repair scope we actually study, and we have added an explicit scope-and-terminology note in Section I.

B. FUTURE WORK

We see several promising directions. (i) Hierarchical memory: layering short-term, mid-term (per-project), and long-term (cross-project) memories may better mirror human expert organization. (ii) Multi-agent LS-CM: combining LS-CM with collaborative role-playing frameworks like ChatDev [13] could yield specialized agents that share a common memory substrate. (iii) End-to-end training: fine-tuning the LLM under LS-CM’s reward signal via DPO [35] could close the gap between symbolic strategy advice and token-level generation. (iv) Standardized tool interfaces: integrating LS-CM’s tool layer with the Model Context Protocol (MCP) [45] which is an emerging open standard for connecting LLM agents to external tools and data sources, would let the same memory and prompt-configuration machinery drive a broad ecosystem of debuggers, build systems, and issue trackers without bespoke per-tool adapters, and would make the STM retrieval layer portable across environments.

C. BROADER APPLICATIONS

The LS-CM paradigm is not specific to software engineering. We see immediate applicability to (a) security vulnerability discovery, where strategies analogous to fuzzing patterns can be learned from CVE databases; (b) autonomous data science, in which long-term memory could store dataset-cleaning heuristics across projects; (c) scientific experimentation, where memory of prior failed experiments could guide hypothesis selection; and (d) interactive theorem proving, where strategic tactic libraries naturally fit the LTM abstraction. We believe that treating memory and prompt structure as first-class learning objects will be a recurring theme as autonomous agents move from demonstrations to dependable deployment.

APPENDIX A

BASE PROMPT TEMPLATE

This appendix presents the initial (un-evolved) prompt configuration corresponding to $\theta_P = (1, 1, 1, 1)$ in our SECE catalog. It serves as the starting point from which the bandit explores alternative variants. The four components are presented separately for clarity; at runtime they are concatenated and supplied to the LLM along with the STM-retrieved context and the LTM strategy advice.

System message ($P_{\text{sys}}^{(1)}$):

You are an expert software engineering agent. Your goal is to resolve a GitHub issue by inspecting the repository, identifying the root cause of the bug, and producing a minimal patch that passes the project’s test suite. Work step by step. At each step, choose a single tool to call. Do not edit files before you have reproduced the failure or confirmed the location of the bug.

Tool description block ($P_{\text{tool}}^{(1)}$):

Available tools:

- view(path, range): inspect a file or directory.
- search(pattern): grep across the repository for a regex pattern.
- edit(path, old, new): replace an exact string in a file.
- run(cmd): execute a shell command.
- test(scope): run the test suite, optionally restricted to a scope.
- submit(): finalize and submit the current patch.

Exemplar block ($P_{\text{exemplar}}^{(1)}$): One worked example of an end-to-end debugging episode on a held-out training instance, demonstrating the expected interleaving of view/run/test/edit calls. Omitted here for space; it will be included verbatim in the artifact released upon acceptance.

Reasoning policy block ($P_{\text{policy}}^{(1)}$):

Reasoning protocol:

1. Restate the issue in your own words.
2. Identify candidate files using STM-retrieved context and the LTM strategy advice provided below.
3. Reproduce the failure before editing.
4. Localize the root cause to a single function when possible.
5. Propose a minimal patch.
6. Re-run the failing tests and all pass-to-pass tests before submitting.

The catalog contains 8 variants of each component (generated offline by GPT-4o rewriting), yielding $8^4 = 4,096$ possible configurations. SECE explores this space using the LinUCB rule of Eq. (6).

APPENDIX B**EXAMPLE OF AN EVOLVED PROMPT**

To illustrate how SECE specializes prompts for different bug contexts, we present a concrete example: the configuration that the bandit converged to for API-deprecation bugs in the django project after 50 training epochs. The selected configuration is $\theta_P = (3, 5, 1, 6)$. Differences from the base configuration of Appendix A are highlighted by the explicit additions shown below.

Evolved system message ($P_{\text{sys}}^{(3)}$):

You are an expert software engineering agent specializing in long-lived Python projects with extensive backward-compatibility guarantees. Many bugs in such projects are caused by API deprecations or signature changes between versions. **Before editing any code, first check the project's changelog and recent commit messages for relevant deprecation notices.** Work step by step.

Evolved tool description block ($P_{\text{tool}}^{(5)}$): identical to the base, but with the additional helper documented:

- `git_log(path, since):` list recent commits that touched a path, with messages. Useful for identifying deprecations.

Evolved reasoning policy block ($P_{\text{policy}}^{(6)}$):

Reasoning protocol:

1. Restate the issue, noting any version numbers or library names mentioned.
2. **Inspect CHANGES.txt, docs/releases, or git_log for recent changes affecting the named symbols.**

TABLE 6. Catalog of LTM strategy templates ($|\Sigma| = 24$).

Strategy Name	One-Line Description
<i>Phase 1: Localization</i>	
LOCATE_VIA_STACK_TRACE	Start from the deepest user-code frame in the stack trace.
LOCATE_VIA_ERROR_MESSAGE	Grep for the verbatim error message across the repository.
LOCATE_VIA_TEST_NAME	Inspect the failing test and its imported modules first.
INSPECT_RECENT_COMMITS	Examine git history for recent changes to candidate files.
TRACE_DATA_FLOW	Follow the data path upstream from the failure point.
TRACE_CALL_GRAPH	Walk the call graph from a known entry point.
<i>Phase 2: Reproduction</i>	
REPRODUCE_THEN_BISECT	Reproduce the failure before any edit; then bisect inputs.
MINIMIZE_TEST_INPUT	Shrink the failing input to the smallest triggering case.
COMPARE_EXPECTED_VS_ACTUAL	Print expected and actual values side-by-side before editing.
ISOLATE_IN_REPL	Reproduce the failure in a minimal REPL session.
<i>Phase 3: Root-Cause Analysis</i>	
INSPECT_TEST_FAILURE	Read the assertion failure carefully before any edit.
CHECK_API_DEPRECATION	Verify whether a used API has been deprecated or renamed.
CHECK_TYPE_CONTRACTS	Verify type annotations and runtime types at the boundary.
CHECK_NULL_EDGE_CASES	Audit handling of None, empty, or zero inputs.
CHECK_BOUNDARY_CONDITIONS	Examine off-by-one and boundary-value handling.
CHECK_CONCURREN	Audit shared state, locks, and ordering assumptions.
<i>Phase 4: Patch Synthesis</i>	
MINIMAL_LOCAL_EDIT	Prefer a localized one-function fix when possible.
FIX_CALL_SITE_NOT_API	Patch the caller rather than the called API.
ADD_GUARD_CLAUSE	Add an early-return or input-validation guard.
REFACTOR_HELPER	Extract a helper to consolidate the fix when duplicated.
REVERT_RECENT_CHANGE	Consider reverting a recent commit when bisection points to it.
<i>Phase 5: Verification</i>	
RUN_FAIL2PASS_FIRST	Run the previously-failing tests before broad regression.
RUN_FULL_REGRESSION	Run the full pass-to-pass suite after a tentative fix.
DOUBLE_CHECK_DIFF	Re-read the diff for unintended changes before submitting.

3. Identify candidate files using STM-retrieved context and LTM advice.

4. Reproduce the failure before editing.
5. Prefer fixing the call site over patching the deprecated API, unless the issue explicitly requests the latter.
6. Re-run the failing tests and all pass-to-pass tests before submitting.

Note that the exemplar block remains the base variant ($P_{\text{exemplar}}^{(1)}$); the SECE bandit found no benefit in replacing it for this bug class. This illustrates a useful property of the per-component credit-assignment of Eq. (7): SECE preserves components that are working and only updates those for which positive counterfactual evidence has accumulated. Across the full SWE-bench Lite split, 62% of the SECE-converged configurations retain at least one base component, indicating that the bandit is conservative rather than indiscriminately exploratory.

APPENDIX C

LTM STRATEGY TEMPLATES

Table 6 lists the complete catalog of $|\Sigma| = 24$ strategic templates used by the LTM module. Each template is a high-level debugging tactic identified from a corpus of human bug-fix commits and expressed as a brief instruction to be injected into the LLM prompt. The derivation of this catalog (commit sampling, two-coder open coding, thematic saturation at 24, and Cohen's $\kappa = 0.74$) is described in Section IV. The LTM policy π_{θ}^{σ} outputs a distribution over this set; the Top-3 strategies are passed to the LLM at inference time. Strategies are grouped by their primary phase of the debugging workflow.

The strategy bank is intentionally biased toward localization and root-cause analysis (Phases 1–3 account for 16/24 entries), reflecting our observation that premature edits are the most common failure mode of prior agents. We did not hand-tune the relative frequency with which the LTM policy invokes each strategy; the PPO procedure of Eq. (4) discovers context-dependent preferences automatically. Post-hoc analysis of the converged policy reveals that LOCATE_VIA_STACK_TRACE, REPRODUCE_THEN_BISECT, and INSPECT_TEST_FAILURE are the three most frequently selected strategies (collectively $\sim 42\%$ of invocations on SWE-bench Lite at convergence; cf. the higher $\sim 60\%$ concentration earlier in training discussed in Section V), while specialized strategies such as CHECK_CONCURRENCY are reserved for specific bug classes (e.g., thread-safety issues in requests). This emergent specialization, learned without explicit supervision over which strategy is appropriate for which bug, is the central empirical evidence that LTM is performing genuine strategic abstraction rather than uniform mixing.

REFERENCES

- [1] D. H. O'Dell, "The debugging mindset," *Queue*, vol. 15, no. 1, pp. 71–90, Feb. 2017.
- [2] T. Britton, L. Jeng, G. Carver, P. Cheak, and T. Katzenellenbogen, "Reversible debugging software: Quantify the time and cost saved using reversible debuggers," Judge Bus. School, Univ. Cambridge, Cambridge, U.K., Tech. Rep. 229, 2013.
- [3] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Trans. Softw. Eng.*, vol. 42, no. 8, pp. 707–740, Aug. 2016.
- [4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, and S. Agarwal, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [5] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
- [6] B. Rozière et al., "Code llama: Open foundation models for code," 2023, *arXiv:2308.12950*.
- [7] R. Li et al., "StarCoder: May the source be with you!" *Trans. Mach. Learn. Res.*, Dec. 2023.
- [8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [10] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 50528–50652.
- [11] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," 2024, *arXiv:2407.01489*.
- [12] S. Hong, Z. Mingchen, C. Jiaqi, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and S. Jürgen, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 23247–23275.
- [13] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, vol. 1, 2024, pp. 15174–15186.
- [14] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.
- [15] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6769–6781.
- [16] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 8634–8652.
- [17] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 46534–46594.
- [18] S. Robertson and H. Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, vol. 4. Boston, MA, USA: Now, 2009.
- [19] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Proc. Findings Assoc. Comput. Linguistics EMNLP*, 2020, pp. 1536–1547.
- [20] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Syatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. B. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "GraphCodeBERT: Pre-training code representations with data flow," in *Proc. Int. Conf. Learn. Represent.*, 2021.
- [21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [22] R. Abreu, P. Zoetewij, and A. J. C. van Gemund, "Spectrum-based multiple fault localization," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng.*, Nov. 2009, pp. 88–99.

- [23] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Haq, A. Sharma, T. Joshi, H. Moazam, H. Miller, and M. Zaharia, "DSPy: Compiling declarative language model calls into state-of-the-art pipelines," in *Proc. Int. Conf. Learn. Represent.*, 2024, pp. 54928–54958.
- [24] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent.*, 2024, pp. 54107–54157.
- [25] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *Proc. Int. Symp. Softw. Test. Anal.*, Jul. 2014, pp. 437–440.
- [26] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "CodeT5: Code generation with generated tests," in *Proc. 11th Int. Conf. Learn. Represent.*, 2022.
- [27] W. Zhao, T. Chen, J. S. Yang, and L. Qiu, "AutoML-pipeline: A RAG-enhanced code generation framework with pre-validation for cloud-native machine learning workflows," *IEEE Access*, vol. 14, pp. 41932–41945, 2026.
- [28] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers Comput. Sci.*, vol. 18, no. 6, Dec. 2024, Art. no. 186345.
- [29] C. Packer, V. Fang, S. Patil, K. Lin, S. Wooders, and J. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023, *arXiv:2310.08560*.
- [30] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 17, pp. 19724–19731.
- [31] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol.*, Oct. 2023, pp. 1–22.
- [32] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Trans. Mach. Learn. Res.*, Mar. 2023.
- [33] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 21314–21328.
- [34] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 27730–27744.
- [35] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 53728–53741.
- [36] X. Liu et al., "AgentBench: Evaluating LLMs as agents," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 52989–53046.
- [37] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning acting and planning in language models," in *Proc. 41st Int. Conf. Mach. Learn.*, 2023, pp. 62138–62160.
- [38] X. Li, W. Li, Y. Zhang, and L. Zhang, "DeepFL: Integrating multiple fault diagnosis dimensions for deep fault localization," in *Proc. 28th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2019, pp. 169–180.
- [39] Y. Li, S. Wang, and T. Nguyen, "Fault localization with code coverage representation learning," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, May 2021, pp. 661–673.
- [40] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy, "InferFix: End-to-end program repair with LLMs," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 1646–1656.
- [41] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT," 2023, *arXiv:2304.00385*.
- [42] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. 19th Int. Conf. World Wide Web*, Apr. 2010, pp. 661–670.
- [43] R. S. Sutton, A. G. Barto, and R. S. Sutton, *Reinforcement Learning: An Introduction*, vol. 1. Cambridge, MA, USA: MIT Press, 1998.
- [44] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [45] Anthropic. (2024). *Introducing the Model Context Protocol*. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>



TIANJUN MO received the M.Eng. degree in computer engineering from Duke University. His work focuses on applying advanced machine learning and LLM techniques to real-world systems, including user verification, adaptive decision modeling, and scalable AI integration in production environments. His research interests include large language models, artificial intelligence systems, human–AI collaboration, intelligent software engineering, multi-agent systems, and AI-driven decision-making.



CHANGHAO ZHANG received the M.S. degree in applied data science from The University of Chicago, USA. His work involves developing predictive models and production scheduling strategies to improve operational efficiency and resource utilization. His research interests include data analytics, machine learning applications, supply chain optimization, and artificial intelligence–driven decision systems.



JITONG ZOU is currently pursuing the Ph.D. degree in computer science with Case Western Reserve University, Cleveland, OH, USA. His work primarily investigates fairness issues in vision-language models and large language models, particularly in safety-critical domains, such as autonomous driving. He is interested in developing empirical evaluation frameworks and data-centric approaches to diagnose and mitigate bias, as well as improving the transparency and reliability of AI systems in real-world deployment. His research interests include responsible artificial intelligence, with a focus on algorithmic fairness and trustworthy machine learning.



ZIXUAN GUO received the M.M. degree in music technology from New York University. His research interests include machine learning, digital signal processing, music information retrieval, symbolic music generation, machine listening, and controllable generative AI for music and audio applications. He has experience in developing machine learning pipelines, audio signal processing systems, low-latency audio applications, and real-time interactive music technologies using Python, C/C++, MATLAB, JUCE, and related frameworks and tools.



MINJAE RHEE received the Master of Computer Science degree from the University of Illinois Urbana–Champaign, USA. She develops and operates large-scale distributed and cloud systems for infrastructure management and reliability. Her research interests include systems engineering, intelligent automation for cloud platforms, deep learning, LLM, distributed computing, and data-driven approaches to performance optimization and system reliability.

...